

Programmazione 1

Corso c

15/10/2023
> Le funzioni

Alessandro Mazzei

alessandro.mazzei@unito.it

Dipartimento di Informatica
Università degli Studi di Torino



UNIVERSITÀ
DI TORINO

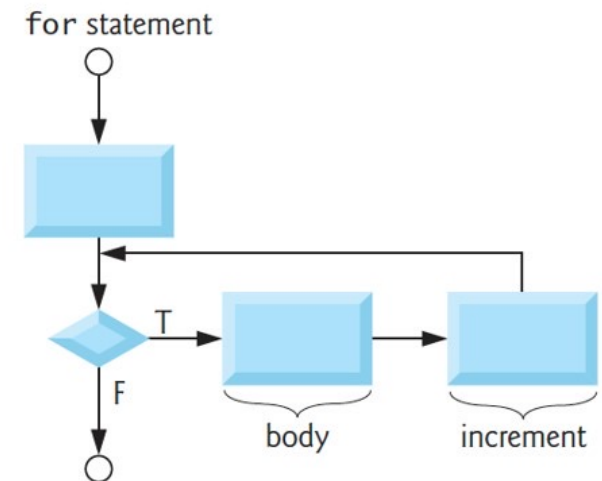
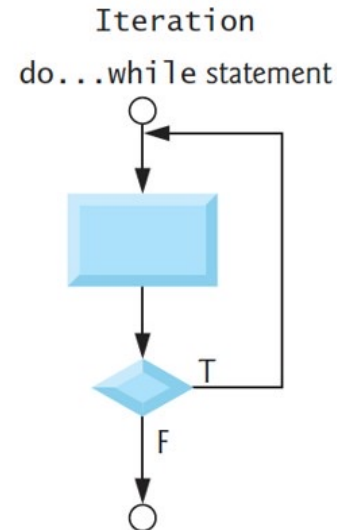
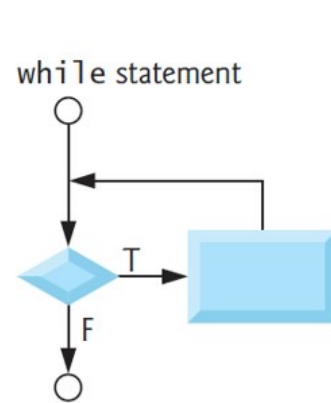
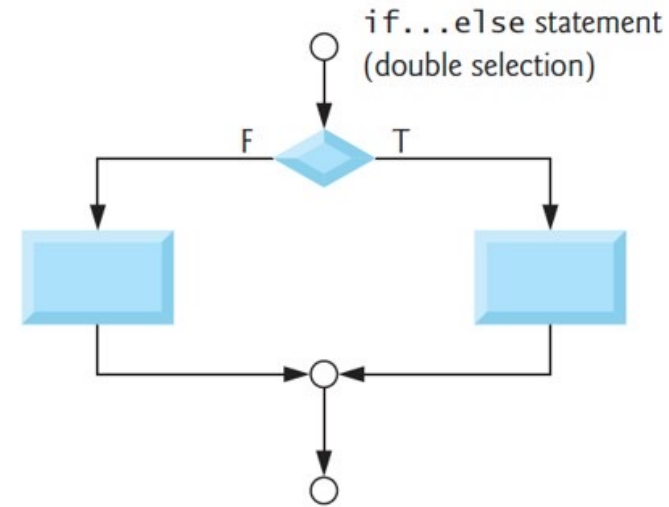
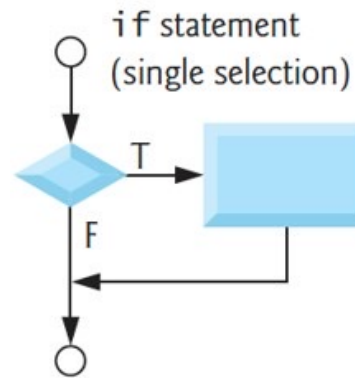
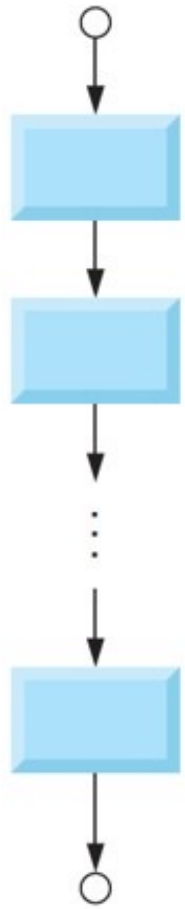
Outline

- Strutture e funzioni
- Modulare: le funzioni di sistema
- Definire nuove funzioni
- Le funzioni e lo stack
- Passaggio per valore vs. riferimento

Outline

- Strutture e funzioni
- Modulare: le funzioni di sistema
- Definire nuove funzioni
- Le funzioni e lo stack
- Passaggio per valore vs. riferimento

Strutture di controllo



Essenza del C

Qualsiasi forma di controllo necessaria in un programma C può essere espressa in termini di tre sole forme di controllo:

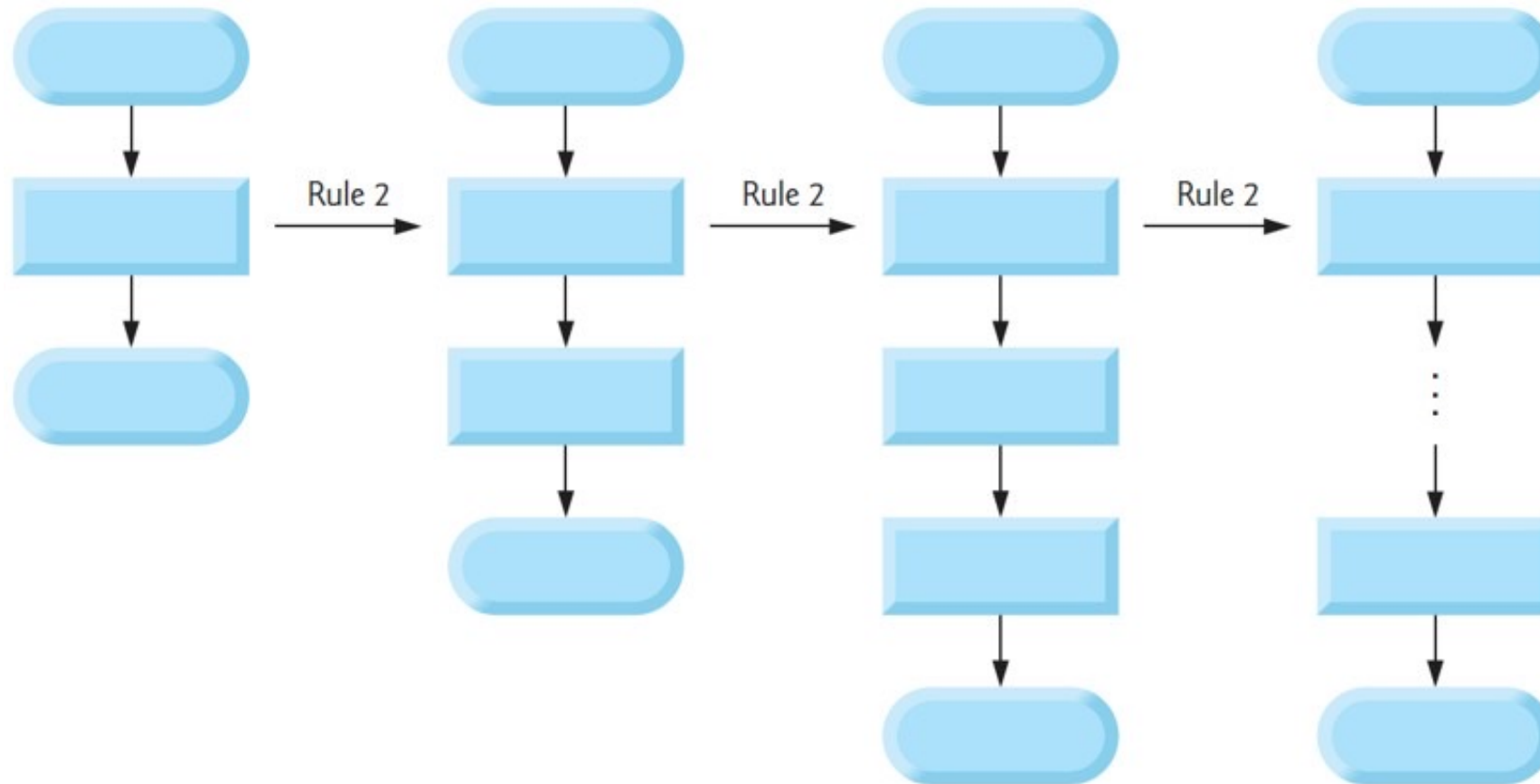
- sequenza
- istruzione if (selezione).
- istruzione while (iterazione).

Queste istruzioni di controllo possono essere combinate solo in due modi: **accatastamento** (stacking) e **annidamento** (nesting).

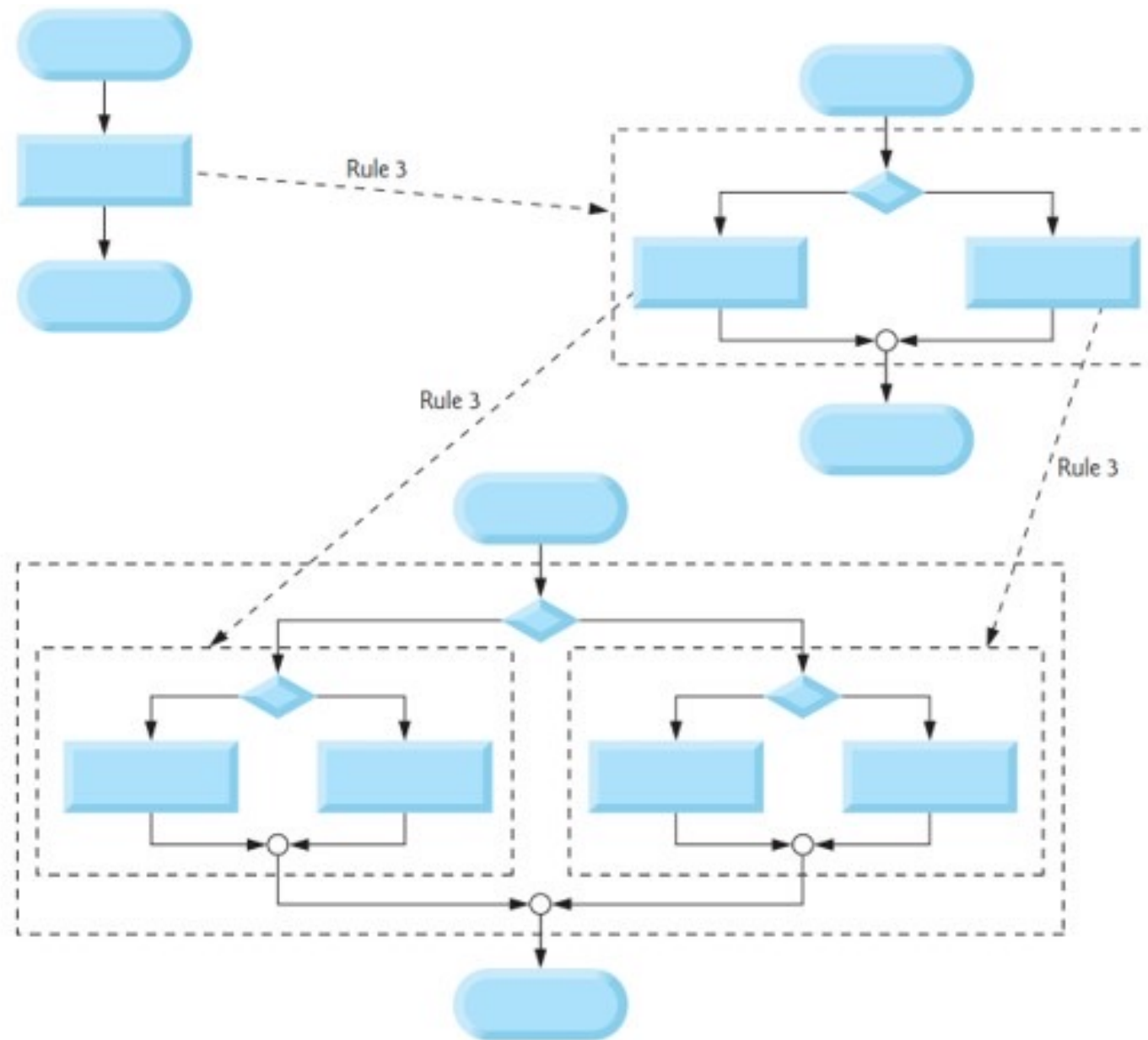
Regole per la creazione di programmi strutturati

1. Iniziate con il "diagramma di flusso più semplice"
2. **Regola dell'impilamento:** ogni rettangolo (azione) può essere sostituito da due rettangoli (azioni) in sequenza.
3. **Regola di annidamento:** ogni rettangolo (azione) può essere sostituito da qualsiasi istruzione di controllo.
4. Le regole 2 e 3 possono essere applicate tutte le volte che si vuole e in qualsiasi ordine.

Stacking



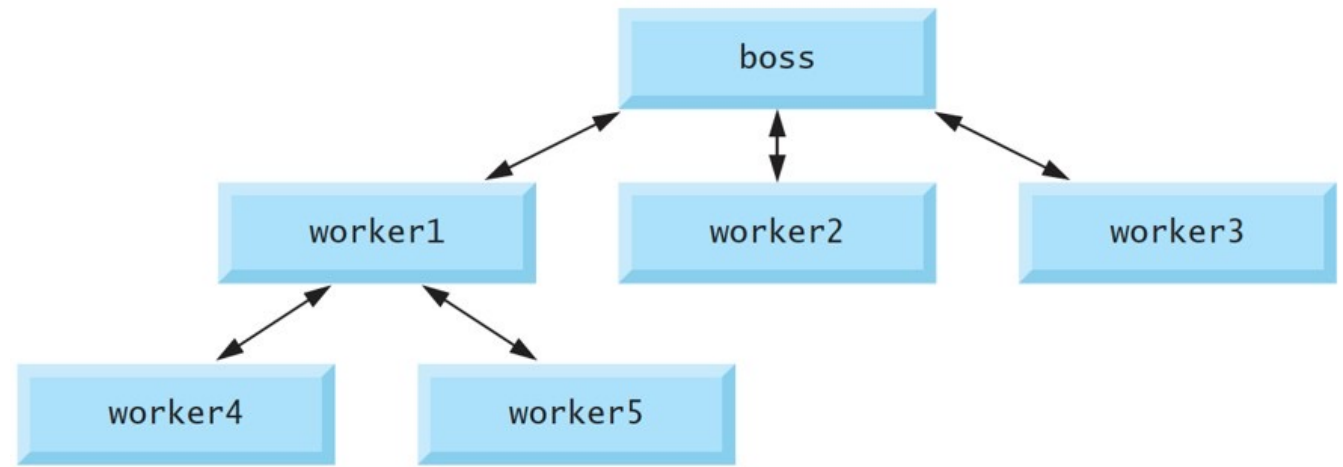
Nesting



Divide et impera

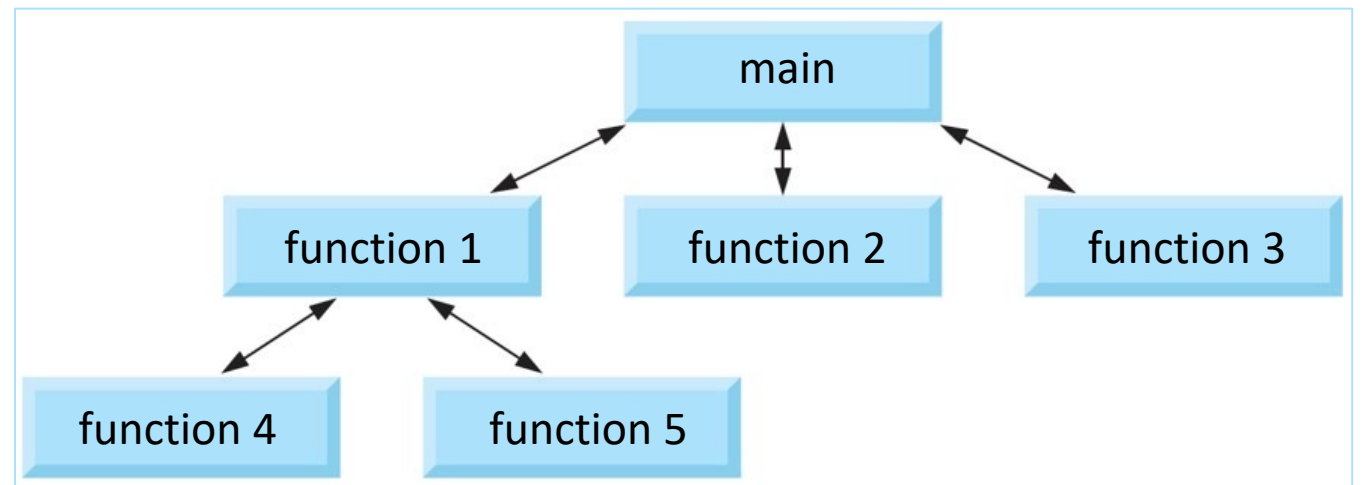
- L'idea della programmazione strutturata si fonde con l'idea della modularizzazione
- Si sviluppano e si mantengono programmi utilizzando pezzi più piccoli, ognuno dei quali è più gestibile del programma originale.
- Le funzioni seguono la stessa idea di modularizzare **blocchi di codice**, parametrizzando il loro funzionamento sulla base di valori di input
- *Software reusability*
- *Abstraction*

Management



Il capo chiede a un lavoratore di eseguire un compito e di riferire al termine dello stesso. Il capo non sa come il lavoratore esegue il compito designato. Il lavoratore può chiamare altri lavoratori e il capo non ne è a conoscenza.

Funzioni



La funzione **main** chiama (invoca) una **funzione** per eseguire un compito e per restituire (eventualmente) un valore al termine dello stesso. La **main** non sa come la funzione esegue il compito commissionato. La **funzione** può chiamare altre funzioni ma la **main** non ne è a conoscenza.

Funzione chiamante  Funzione chiamata

Outline

- Strutture e funzioni
- Modulare: le funzioni di sistema
- Definire nuove funzioni
- Le funzioni e lo stack
- Passaggio per valore vs. riferimento

Funzioni pre-definite: libreria standard

- Le funzioni della libreria matematica (header `math.h`) eseguono i calcoli matematici più comuni
- `printf("%.2f", sqrt(900.0)) ;`
 - Richiama la funzione `sqrt` per calcolare la radice quadrata di 900,0, quindi stampa il risultato come 30,00.
 - `sqrt` accetta un argomento `double` e restituisce un risultato `double`
 - Tutte le funzioni della libreria matematica che restituiscono valori in virgola mobile restituiscono il tipo `double`.
- Gli argomenti delle funzioni possono essere costanti, variabili o *espressioni*.
- La tabella seguente riassume alcune funzioni della libreria matematica del C:
 - Nella tabella, le variabili `x` e `y` sono di tipo `double`
 - Il C11 ha aggiunto le funzionalità per i numeri complessi tramite l'intestazione `complex.h`

math.h

Function	Description	Example
<code>sqrt (x)</code>	square root of x	<code>sqrt (900.0)</code> is 30.0 <code>sqrt(9.0)</code> is 3.0
<code>cbrt (x)</code>	cube root of x (C99 and C11 only)	<code>cbrt (27.0)</code> is 3.0
<code>exp (x)</code>	exponential function e^x	<code>exp (1.0)</code> is 2.718282 <code>exp (2.0)</code> is 7.389056
<code>log (x)</code>	natural logarithm of x (base e)	<code>log (2.718282)</code> is 1.0 <code>log (7.389056)</code> is 2.0
<code>log10 (x)</code>	logarithm of x (base 10)	<code>log10 (1.0)</code> is 0.0 <code>log10 (10.0)</code> is 1.0 <code>log10 (100.0)</code> is 2.0
<code>fabs (x)</code>	absolute value of x as a floating-point number	<code>fabs (13.5)</code> is 13.5 <code>fabs (0.0)</code> is 0.0

math.h

Function	Description	Example
<code>ceil(x)</code>	rounds x to the smallest integer not less than x	<code>ceil(9.2)</code> is 10.0
<code>floor(x)</code>	rounds x to the largest integer not greater than x	<code>floor(9.2)</code> is 9.0
<code>pow(x, y)</code>	x raised to power ____	<code>pow(2, 7)</code> is 128.0 <code>pow(9, .5)</code> is 3.0
<code>fmod(x, y)</code>	remainder of _____ as a floating-point number	<code>fmod(13.657, 2.333)</code> is 1.992
<code>sin(x)</code>	trigonometric sine of x (x in radians)	<code>sin(0.0)</code> is 0.0
<code>cos(x)</code>	trigonometric cosine of x (x in radians)	<code>cos(0.0)</code> is 1.0
<code>tan(x)</code>	trigonometric tangent of x (x in radians)	<code>tan(0.0)</code> is 0.0

Outline

- Strutture e funzioni
- Modulare: le funzioni di sistema
- Definire nuove funzioni
- Le funzioni e lo stack
- Passaggio per valore vs. riferimento

Programmazione modulare

- Finora abbiamo scritto tutto il nostro codice nella funzione *main()*
- Alcuni algoritmi come lo scambio di variabili sono richiesti più volte nel programma
- Abbiamo copiato il blocco di codice dello scambio variabili dove richiesto
- Codice identico in tre punti del programma, cambia solo nome delle variabili

```
...  
if (z < x) {  
    temp = z;  
    z = x;  
    x = temp;  
}  
if (z < y) {  
    temp = z;  
    z = y;  
    y = temp;  
}  
if (y < x) {  
    temp = y;  
    y = x;  
    x = temp;  
}  
...
```

Programmazione modulare

- Copiare lo stesso codice è un cattivo approccio per diverse ragioni:
 - Compattezza del codice
 - Manutenibilità del software
 - Riutilizzabilità del codice
 - Modularità dello sviluppo

```
...  
if (z < x) {  
    temp = z;  
    z = x;  
    x = temp;  
}  
if (z < y) {  
    temp = z;  
    z = y;  
    y = temp;  
}  
if (y < x) {  
    temp = y;  
    y = x;  
    x = temp;  
}  
...
```

Funzione

Una funzione è un **blocco di istruzioni** a cui do un *nome*,
che può accettare degli *input* e può produrre degli *output*

Le funzioni

- Finora tutto il nostro codice si trovava nella funzione *main()*
- Le funzioni rappresentano un blocco di codice invocabile dall'esterno con *parametri formali di input* e *valore di ritorno*

`y = square(x);`

y è il valore ritornato *x* è il parametro

- Possono accettare in input zero o più **parametri**

```
printf("%d diviso %d: %d\n", x, y, x/y);
```

- Possono non restituire alcun valore

```
scanf("%d", &x);
```

Le funzioni

- Il *prototipo* della funzione viene **dichiarato** prima del `main()`
 - Permette al compilatore controlli correttezza numero e tipo parametri input e su tipo di valore ritornato
- La funzione viene **definita** dopo la funzione *main()*
 - La definizione di funzione deve essere concorde con il prototipo per numero e per tipo di parametri di input e tipo di parametro di ritorno
- Valgono le regole viste per la **nomomenclatura** degli identificatori
 - nome inizia per minuscolo, niente spazi, camelToe convention
 - una sola funzione con un dato nome

square

```
1. // fig05_01.c
2. // Creazione e uso di una funzione.
3. #include <stdio.h>
4.
5. int square(int number);
6.
7. int main(void) {
8.
9.     for (int x = 1; x <= 10; ++x) {
10.         printf("%d  ", square(x));
11.     }
12.
13.     puts("");
14. }
15.
16.
17. int square(int number) {
18.     return number * number;
19. }
```

square

```
1. // fig05_01.c
2. // Creazione e uso di una funzione.
3. #include <stdio.h>
4.
5. int square(int number); // prototipo di funzione
6.
7. int main(void) {
8.     // ripeti 10 volte e ogni volta calcola e stampa il quadrato di x
9.     for (int x = 1; x <= 10; ++x) {
10.         printf("%d  ", square(x)); // chiamata della funzione
11.     }
12.
13.     puts("");
14. }
15.
16. // la definizione di square restituisce il quadrato del suo parametro
17. int square(int number) { //number è una copia dell'arg. della funzione
18.     return number * number; //restituisce il quadrato di number come int
19. }
```

square

```
1. // fig05_01.c
2. // Creazione e uso di una funzione.
3. #include <stdio.h>
4.
5. int square(int number); // prototipo di funzione
6.
7. int main(void) {
8.     // ripeti 10 volte e ogni volta calcola e stampa il quadrato di x
9.     for (int x = 1; x <= 10; ++x) {
10.        printf("%d  ", square(x)); // chiamata della funzione
11.    }
12.
13.    puts("");
14.}
15.
16. // la definizione di square restituisce il quadrato del suo parametro
17. int square(int number) { // number è una copia dell'arg. della funzione
18.     return number * number; // restituisce il quadrato di number come int
19. }
```

Dichiarazione
del prototipo

Chiamata della
funzione

Definizione della
funzione

square

- Chiamata
- Definizione
- Variabili locali
- Prototipo
- Corpo
- Return

```
1. // fig05_01.c
2. // Creazione e uso di una funzione.
3. #include <stdio.h>
4.
5. int square(int number);
6.
7. int main(void) {
8.
9.     for (int x = 1; x <= 10; ++x) {
10.         printf("%d  ", square(x));
11.     }
12.
13.     puts("");
14. }
15.
16.
17. int square(int number) {
18.     return number * number;
19. }
```

square

- Chiamata
- Definizione
- Variabili locali
- Prototipo
- Corpo
- Return

```
1. // fig05_01.c
2. // Creazione e uso di una funzione.
3. #include <stdio.h>
4.
5. int square(int number);
6.
7. int main(void) {
8.
9.     for (int x = 1; x <= 10; ++x) {
10.         printf("%d ", square(x));
11.     }
12.
13.     puts("");
14. }
15.
16.
17. int square(int number) {
18.     return number * number;
19. }
```

- La funzione square è **invocata** o **chiamata** nella funzione main all'interno dell'istruzione printf (riga 10): `printf("%d ", square(x));`
- La funzione square riceve una copia del valore dell'argomento x nel parametro number (riga 17). Poi square calcola `number * number` e restituisce il risultato alla riga 10 nella funzione main dove square è stata invocata.
- La riga 10 passa il risultato di square alla funzione printf, che stampa il risultato sullo schermo. Questo processo è ripetuto 10 volte, una volta per ogni iterazione dell'istruzione for.

square

```
1. // fig05_01.c
2. // Creazione e uso di una funzione.
3. #include <stdio.h>
4.
5. int square(int number);
6.
7. int main(void) {
8.
9.     for (int x = 1; x <= 10; ++x) {
10.         printf("%d  ", square(x));
11.     }
12.
13.     puts("");
14. }
15.
16.
17. int square(int number) {
18.     return number * number;
19. }
```

- Chiamata
- Definizione
- Variabili locali
- Prototipo
- Corpo
- Return

- La definizione della funzione `square` (righe 17–19) mostra che essa si aspetta un parametro intero `number`. La parola chiave `int` che precede il nome della funzione (riga 17) indica che `square` restituisce un risultato intero. L'istruzione `return` in `square` restituisce il risultato di `number * number` alla funzione chiamante.
- La scelta di nomi di funzioni e parametri significativi rende i programmi più leggibili e aiuta a evitare un uso eccessivo di commenti. I programmi dovrebbero essere scritti come raccolte di piccole funzioni. Ciò semplifica la scrittura, il debugging, la manutenzione, la modifica e il riutilizzo dei programmi.
- Una funzione che richiede un numero elevato di parametri potrebbe eseguire troppe attività, pertanto considerate di dividerla in funzioni più piccole che eseguano le attività separatamente. Il tipo di ritorno, il nome e la lista dei parametri della funzione dovrebbero essere contenuti su una singola riga, se possibile.

Formato della definizione di una funzione

```
tipo-del-valore-di-ritorno nome-della-funzione (lista-dei-parametri) {  
    istruzioni  
}
```

- La lista-dei-parametri è un elenco separato da virgole che specifica i parametri che la funzione riceve quando è chiamata. Se una funzione non riceve alcun valore, la lista-dei-parametri dovrebbe contenere la parola chiave `void`. Ogni parametro deve includere il suo tipo; altrimenti, si verifica un errore di compilazione.
- Sebbene non sia scorretto fare così, non usate gli stessi nomi per gli **argomenti** di una funzione e per i corrispondenti **parametri** nella definizione della funzione; ciò contribuisce a evitare ambiguità.
- parametro formale vs. parametro attuale

square

- Chiamata
- Definizione
- Variabili locali
- Prototipo
- Corpo
- Return

```
1. // fig05_01.c
2. // Creazione e uso di una funzione.
3. #include <stdio.h>
4.
5. int square(int number);
6.
7. int main(void) {
8.
9.     for (int x = 1; x <= 10; ++x) {
10.         printf("%d  ", square(x));
11.     }
12.
13.     puts("");
14. }
15.
16.
17. int square(int number) {
18.     return number * number;
19. }
```

- Tutte le variabili definite nelle definizioni di funzione sono **variabili locali**: è possibile accedervi solo all'interno della funzione nella quale sono definite. La maggior parte delle funzioni ha parametri che permettono la comunicazione tra funzioni tramite argomenti nelle chiamate delle funzioni. **I parametri di una funzione sono anche variabili locali di quella funzione.**

square

```
1. // fig05_01.c
2. // Creazione e uso di una funzione.
3. #include <stdio.h>
4.
5. int square(int number);
6.
7. int main(void) {
8.
9.     for (int x = 1; x <= 10; ++x) {
10.         printf("%d  ", square(x));
11.     }
12.
13.     puts("");
14. }
15.
16.
17. int square(int number) {
18.     return number * number;
19. }
```

- Chiamata
- Definizione
- Variabili locali
- Prototipo
- Corpo
- Return

- La riga 5 `int square(int number);` è un **prototipo di funzione**. L'`int` tra parentesi informa il compilatore che `square` si aspetta di ricevere un valore intero dalla funzione chiamante. L'`int` alla sinistra del nome della funzione `square` informa il compilatore che `square` restituisce alla funzione chiamante un risultato intero.
- Il compilatore confronta la chiamata di `square` (riga 10) al suo prototipo per accertarsi che:
 - il numero di argomenti sia corretto;
 - i tipi degli argomenti siano corretti;
 - i tipi degli argomenti siano in ordine corretto;
 - il tipo di ritorno sia coerente con il contesto nel quale è chiamata la funzione.
- Il prototipo della funzione, la prima riga della definizione della funzione e le chiamate della funzione dovrebbero concordare nel numero, nel tipo e nell'ordine degli argomenti e dei parametri. Il prototipo della funzione e l'intestazione della funzione devono avere lo stesso tipo di ritorno.

square

- Chiamata
- Definizione
- Variabili locali
- Prototipo
- Corpo
- Return

```
1. // fig05_01.c
2. // Creazione e uso di una funzione.
3. #include <stdio.h>
4.
5. int square(int number);
6.
7. int main(void) {
8.
9.     for (int x = 1; x <= 10; ++x) {
10.         printf("%d  ", square(x));
11.     }
12.
13.     puts("");
14. }
15.
16.
17. int square(int number) {
18.     return number * number;
19. }
```

- Le istruzioni all'interno delle parentesi graffe formano il corpo della funzione, che è anche detto blocco. Le variabili locali possono essere dichiarate in qualunque blocco e i blocchi possono essere annidati. Le funzioni non possono essere annidate: definire una funzione all'interno di un'altra funzione è un errore di sintassi.

square

```
1. // fig05_01.c
2. // Creazione e uso di una funzione.
3. #include <stdio.h>
4.
5. int square(int number);
6.
7. int main(void) {
8.
9.     for (int x = 1; x <= 10; ++x) {
10.         printf("%d  ", square(x));
11.     }
12.
13.     puts("");
14. }
15.
16.
17. int square(int number) {
18.     return number * number;
19. }
```

- Chiamata
- Definizione
- Variabili locali
- Prototipo
- Corpo
- Return

- Vi sono tre modi per restituire il controllo da una funzione chiamata al punto in cui tale funzione è stata invocata.
- 1. Se la funzione non restituisce alcun risultato, il controllo viene semplicemente restituito quando viene raggiunta la parentesi graffa destra che termina la funzione
- 2. o quando si esegue l'istruzione
 return;
- 3. se la funzione restituisce un risultato, l'istruzione
 return espressione;
restituisce alla funzione chiamante il valore di espressione..

Cosa restituisce `main`?

- Il valore di ritorno `int` di `main` indica se il programma è stato eseguito correttamente.
- Lo standard C indica che `main` restituisce implicitamente 0 se si omette l'istruzione `return` alla fine di `main`.
- È possibile restituire esplicitamente valori non nulli da `main` per indicare che si è verificato un problema durante l'esecuzione del programma.

Ritornare un intero dal main

```
int square(int a);

int main(void) {
    int x=0, x2=0;
    printf ("Inserire intero: ");
    scanf ("%d", &x);
    x2 = square(x);

    return x2;
}

int square(int a) {
    int ret = a * a;
    return ret;
}
```

Nella shell bash il valore ritornato dal programma terminato è accessibile come con la variabile di sistema \$?



```
$ ./square
Inserire intero: 10
$ echo "Risultato: $?"
Risultato: 100
```

Esercizio: funzione maximum

- Massimo di 2 interi
- Massimo di 3 interi

- *Prototipi anonimi*
- *Quanti return?* **SOLO 1**

Outline

- Strutture e funzioni
- Modulare: le funzioni di sistema
- Definire nuove funzioni
- Le funzioni e lo stack
- Passaggio per valore vs. riferimento

<https://pythontutor.com/c.html#mode=edit>

```
#include <stdio.h>

int square(int number);

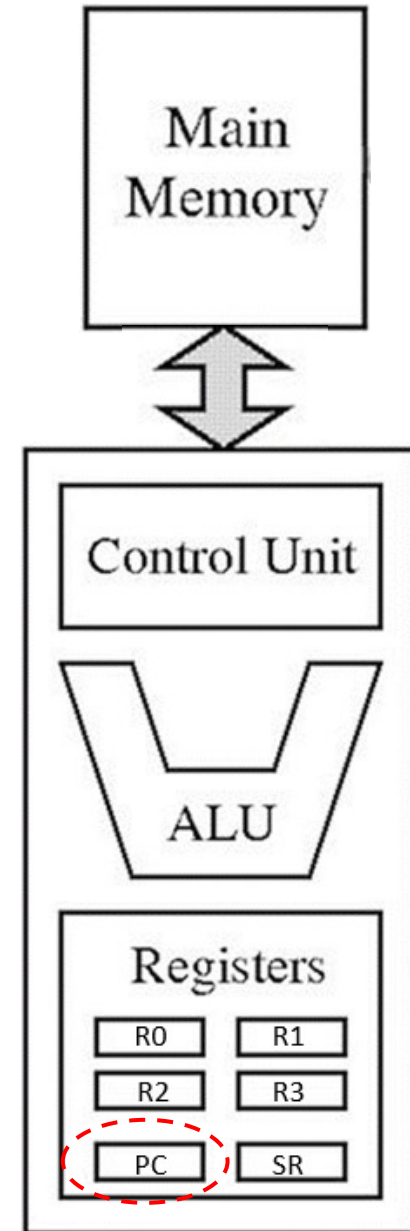
int main(void) {
    int res;
    for (int x = 1; x <= 10; ++x) {
        res = square(x);
        printf("%d^2=%d\n", x, res);
    }
}

int square(int number) {
    int ret = number * number;

    return ret;
}
```

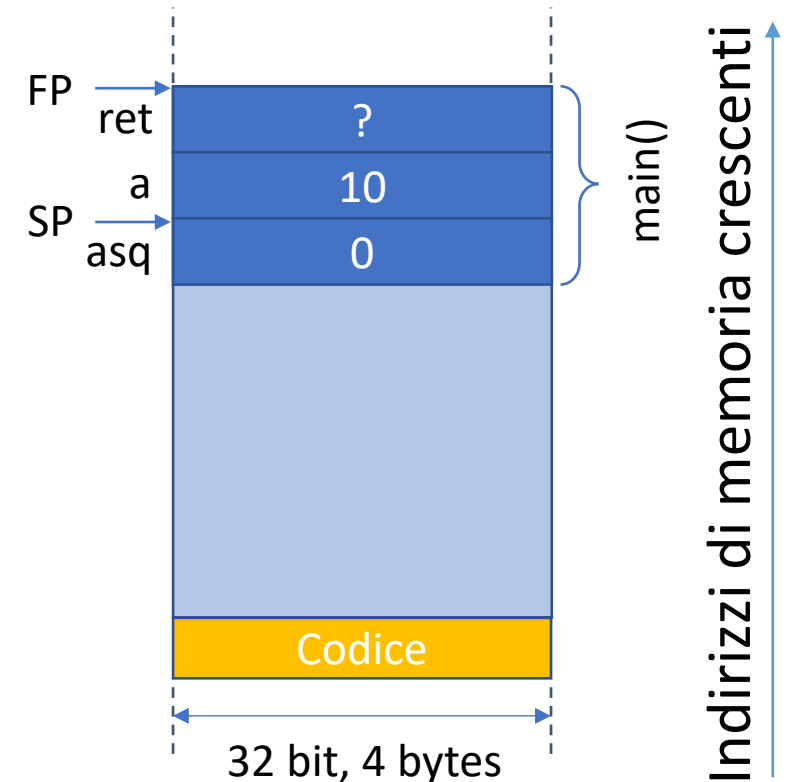
MdV e PC

- Il Program Counter (PC) contiene l'indirizzo nella memoria programma della **prossima** istruzione da eseguire
- All'accensione, vale 0
- Alla fine di ogni istruzione viene incrementato di una parola (es, 4 bytes)
- Le istruzioni di salto JMP JEQ, etc. semplicemente impostano il PC all'indirizzo della linea a cui saltare



Chiamata a funzione e stack

- Ogni chiamata a funzione comporta l'allocazione di un nuovo *frame* (*record di attivazione*) nella memoria stack
- La funzione main ha un suo frame nello stack
- L'indirizzo della prima parola del frame è memorizzato nel registro **Frame Pointer (FP)**
- L'indirizzo dell'ultima parola del frame è memorizzato nel registro **Stack Pointer (SP)**
- Lo stack cresce verso il basso, quindi l'indirizzo dello SP è minore o uguale all'indirizzo del FP



Chiamata a funzione e frame

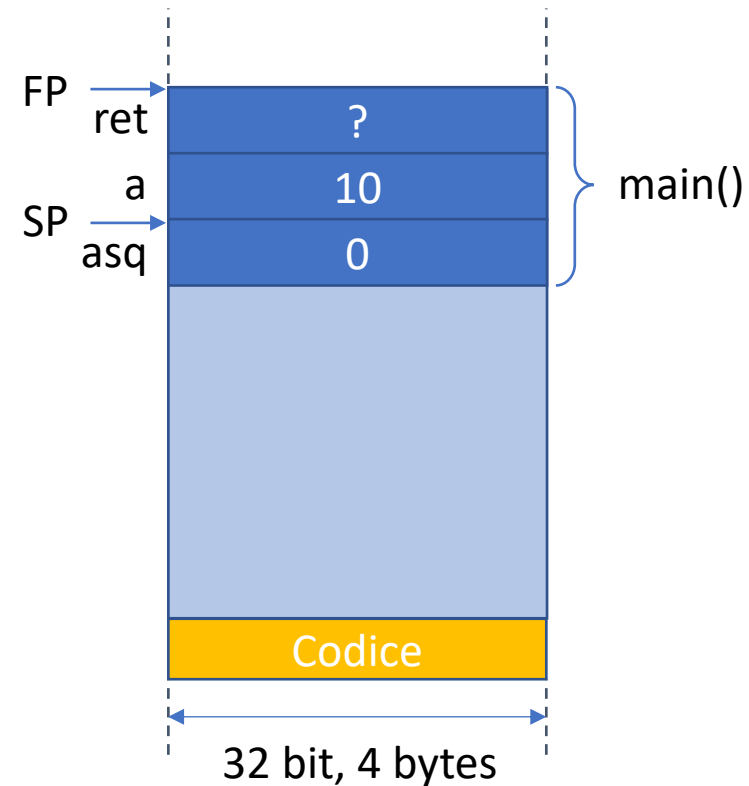
Il frame allocato per la funzione chiamata contiene:

1. una cella per l'indice della *prossima* linea di codice da eseguire dopo il ritorno della funzione chiamata (*retl*)
2. una cella per il valore restituito dalla funzione chiamata (*retv*)
3. una cella per ogni **parametro** della funzione chiamata
4. una cella per ogni **eventuale variabile locale** della funzione chiamata

Chiamata a funzione *square()*

```
1 void main(void) {  
2     int a = 10, asq = 0;  
3     asq = square (a);  
4     printf("%d squared:  
5 %d\n", a, asq);  
6 }  
7  
8 int square (int x) {  
9     int x2 = x * x;  
10    return x2;  
11 }
```

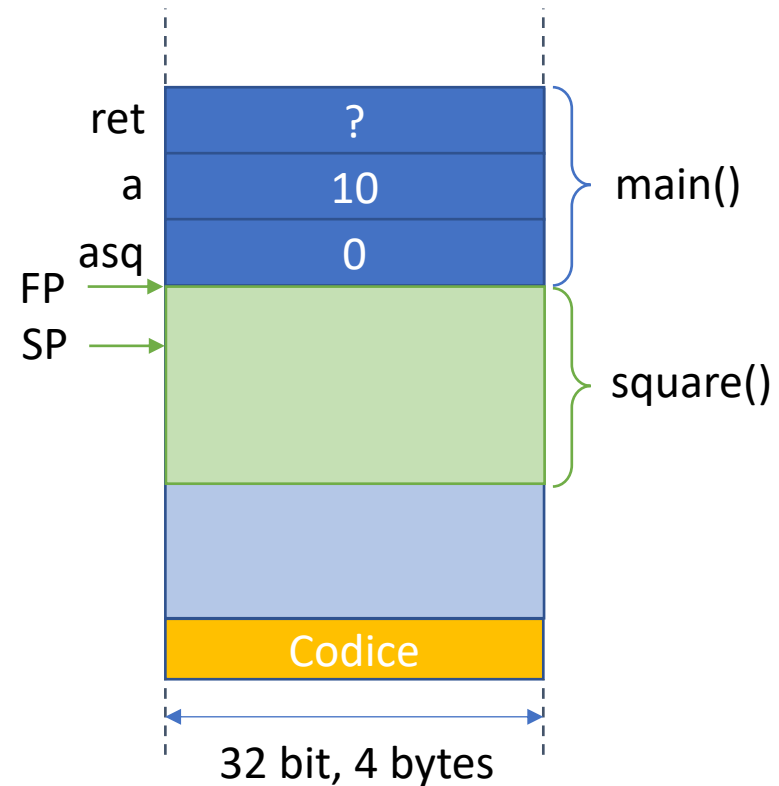
main ha allocato *a* ed *asq* nello stack e sta per chiamare *square()*



Indirizzi di memoria crescenti ↑

Chiamata a funzione *square()*

```
1 void main(void) {  
2     int a = 10, asq = 0;  
3     asq = square (a);  
PC → 4     printf("%d squared:  
      %d\n", a, asq);  
5 }  
6 int square (int x) {  
7     int x2 = x * x;  
8     return x2;  
9 }
```



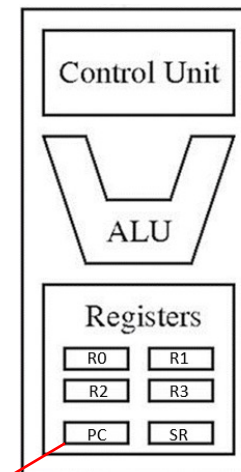
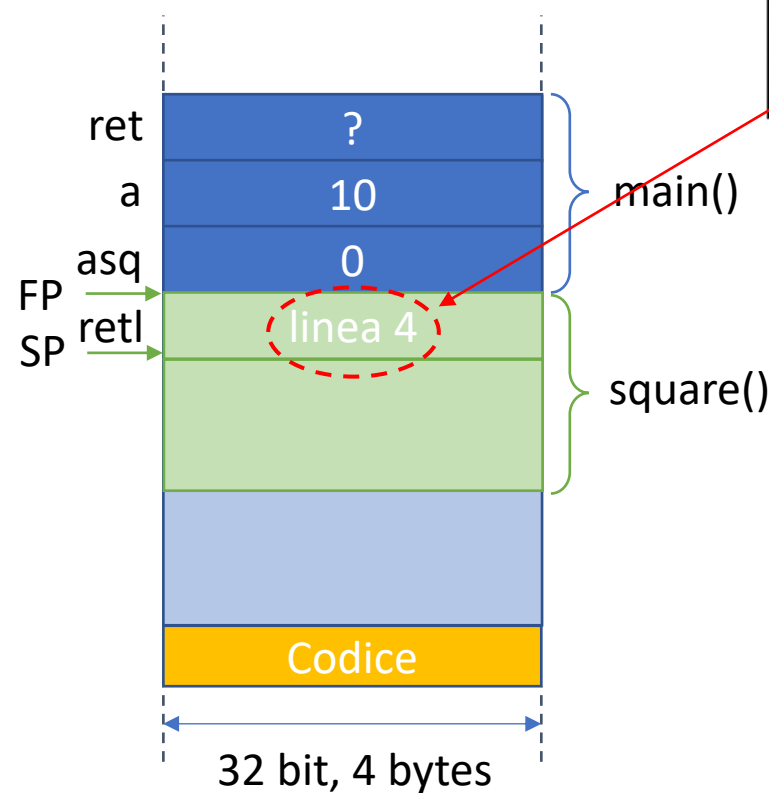
Indirizzi di memoria crescenti ↑

Viene allocato il frame di *square()*
formato da 4 celle in questo esempio

Chiamata a funzione *square()*

```
1 void main(void) {  
2     int a = 10, asq = 0;  
3     asq = square (a);  
4     printf("%d squared:  
5     %d\n", a, asq);  
6 }  
7  
8 int square (int x) {  
9     int x2 = x * x;  
10    return x2;  
11 }
```

PC →

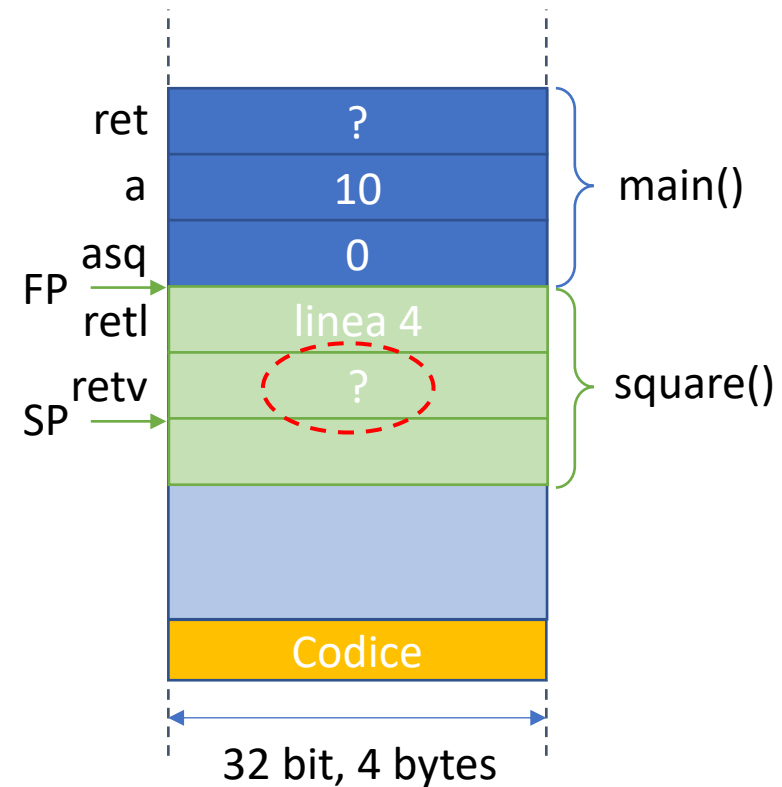


Indirizzi di memoria crescenti ↑

L'indirizzo di ritorno, la linea 4, è salvato nello stack per primo come *retl*

Chiamata a funzione *square()*

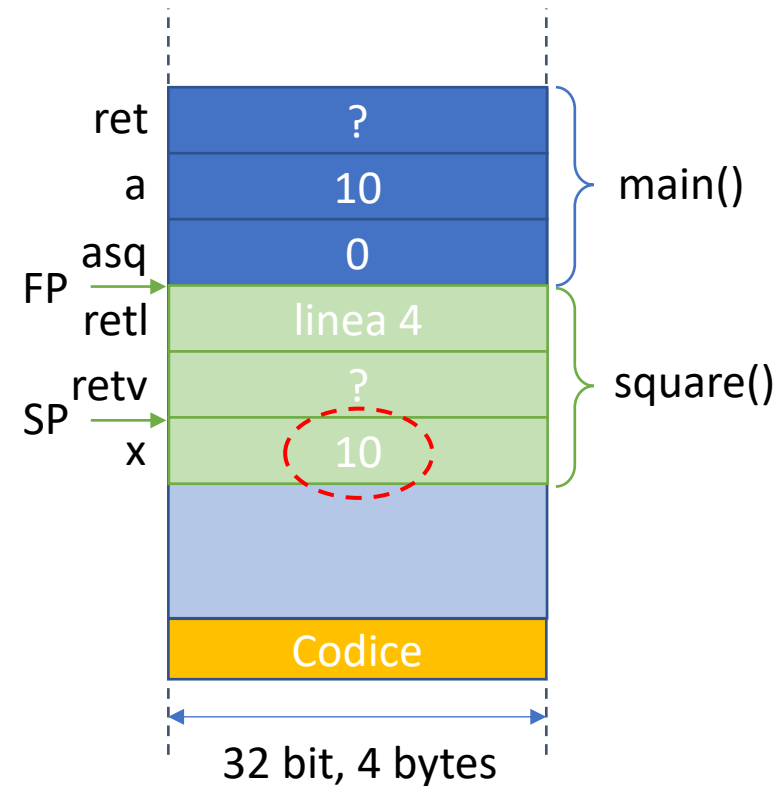
```
1 void main(void) {
2     int a = 10, asq = 0;
3     asq = square (a);
4     printf("%d squared:
5 %d\n", a, asq);
6 }
7 int square (int x) {
8     int x2 = x * x;
9     return x2;
10 }
```



Il valore di ritorno di *square()* sarà salvato nella parola *retv*, per ora è ignoto

Chiamata a funzione *square()*

```
1 void main(void) {  
2     int a = 10, asq = 0;  
3     asq = square (a);  
4     printf("%d squared:  
5 %d\n", a, asq);  
6 }  
7  
8 int square (int x) {  
9     int x2 = x * x;  
10    return x2;  
11 }
```

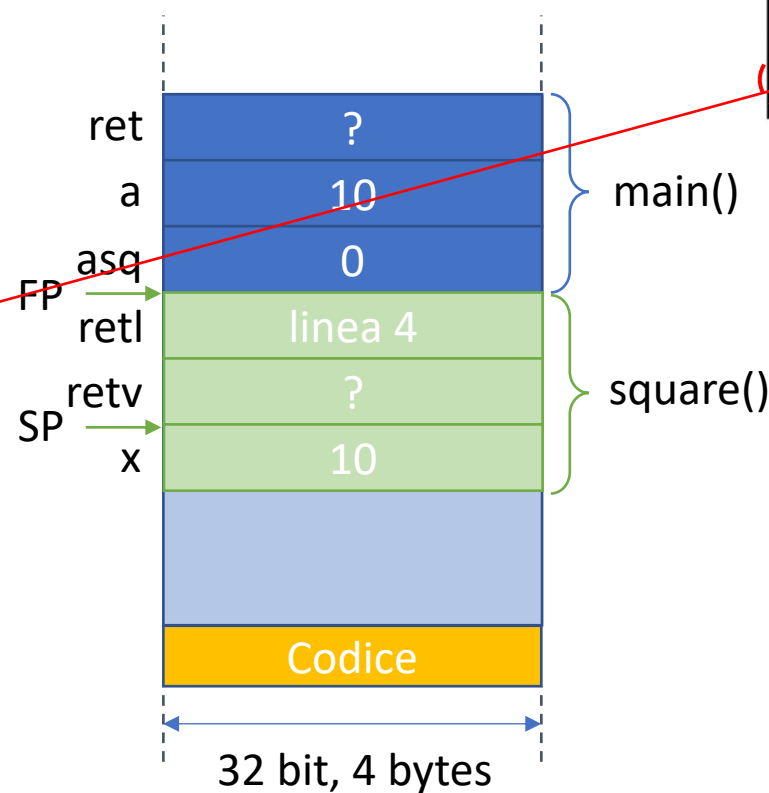


Il parametro *x* di *square()* vale 10

Chiamata a funzione *square()*

```
1 void main(void) {  
2     int a = 10, asq = 0;  
3     asq = square (a);  
4     printf("%d squared:  
5 %d\n", a, asq);  
6 }  
7 int square (int x) {  
8     int x2 = x * x;  
9     return x2;  
}
```

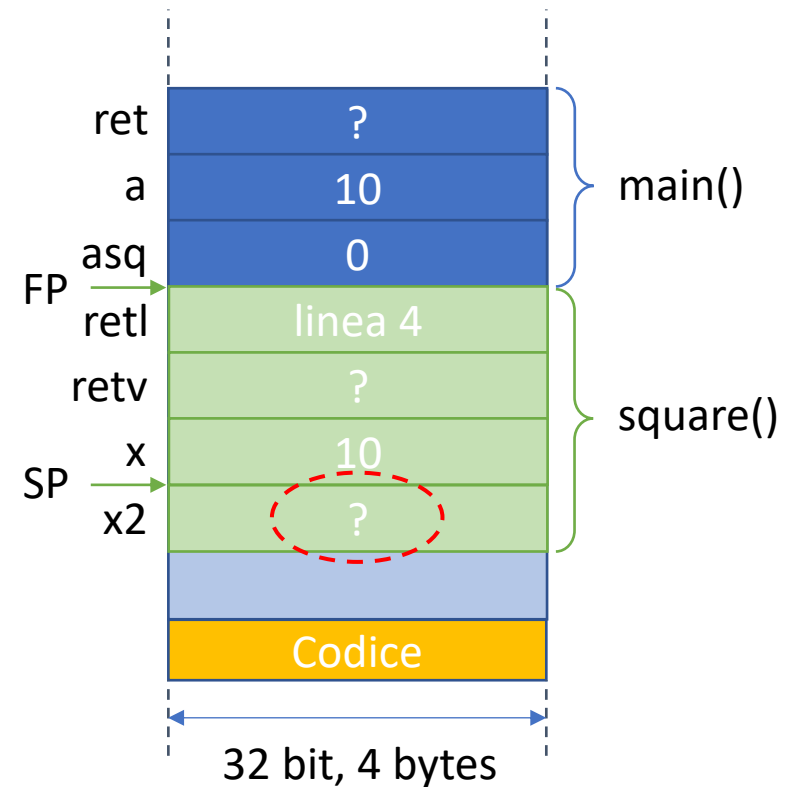
PC →



Il PC è aggiornato alla prima linea di *square()*, cioè la linea 7

Chiamata a funzione *square()*

```
1 void main(void) {  
2     int a = 10, asq = 0;  
3     asq = square (a);  
4     printf("%d squared:  
5 %d\n", a, asq);  
6 }  
7  
8 int square (int x) {  
9     int x2 = x * x;  
10    return x2;  
11 }
```

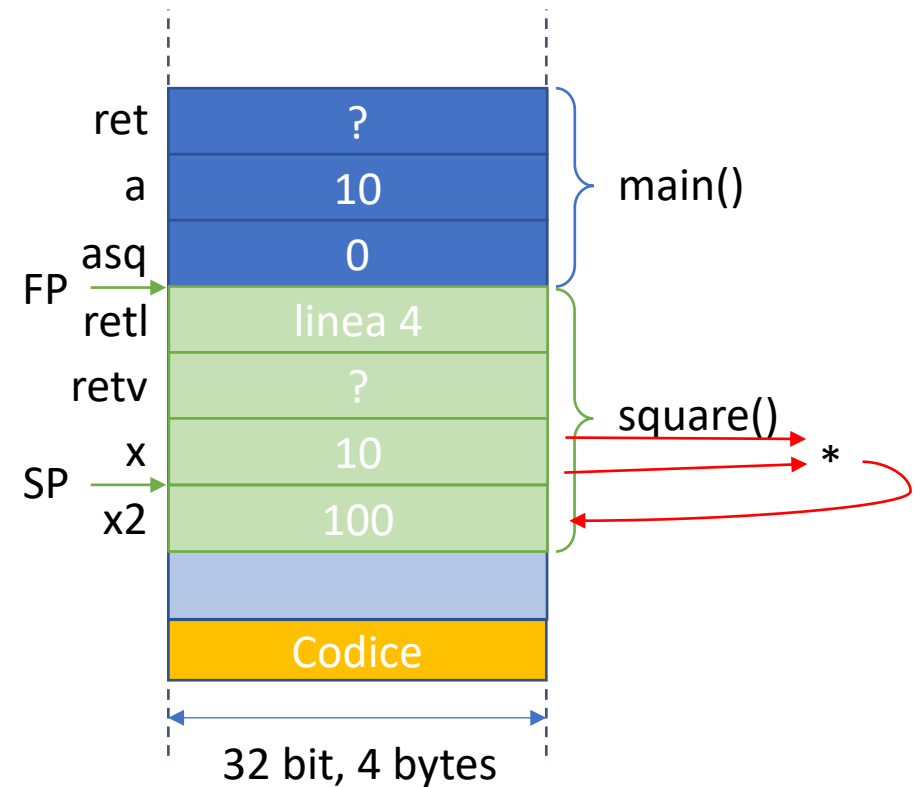


Viene allocata *x2* nel frame di *square()*

Indirizzi di memoria crescenti ↑

Chiamata a funzione *square()*

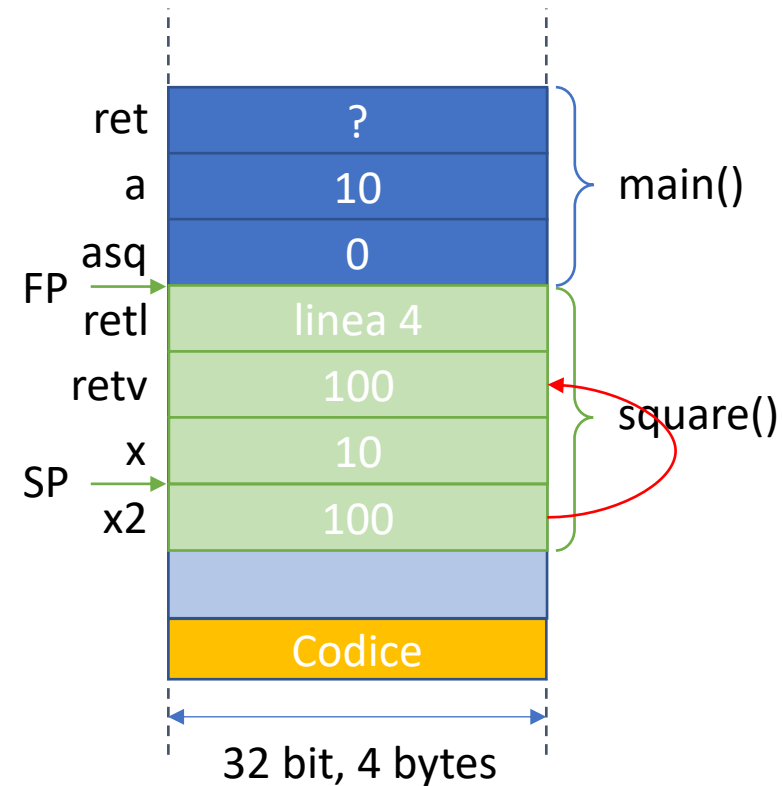
```
1 void main(void) {  
2     int a = 10, asq = 0;  
3     asq = square (a);  
4     printf("%d squared:  
5 %d\n", a, asq);  
6 }  
7  
8 int square (int x) {  
9     int x2 = x * x;  
10    return x2;  
11 }
```



Viene calcolato il valore di $x2 = x * x$

Chiamata a funzione *square()*

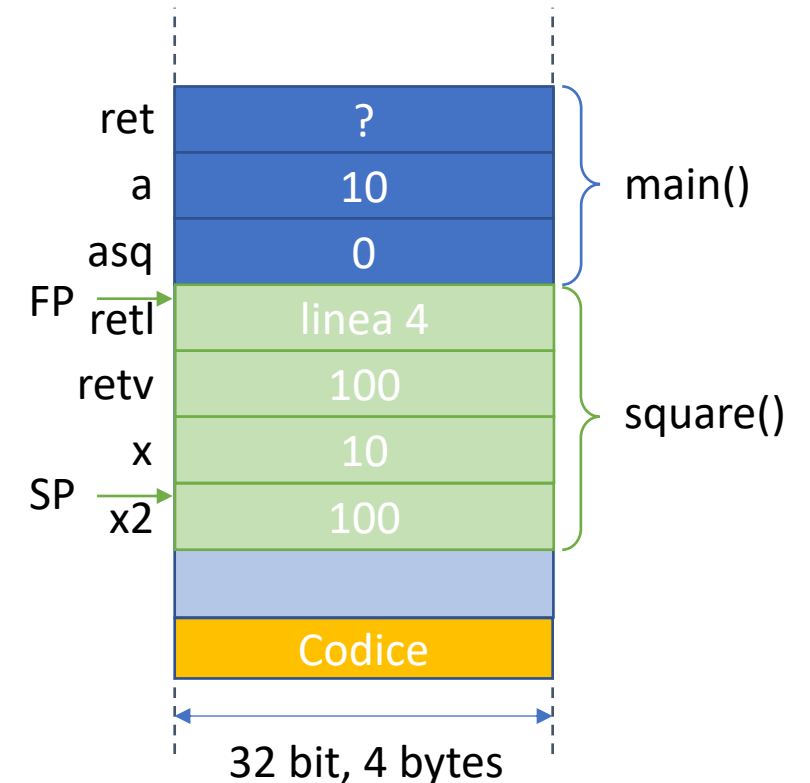
```
1 void main(void) {  
2     int a = 10, asq = 0;  
3     asq = square (a);  
4     printf("%d squared:  
5 %d\n", a, asq);  
6 }  
7  
8 int square (int x) {  
9     int x2 = x * x;  
10    return x2;  
11 }
```



Return comporta la copia di x2 in vret e
l'avvio del ritorno da chiamata

Chiamata a funzione *square()*

- Prima del ritorno dalla funzione chiamata *square()* al *main()*, la situazione è quella a destra
- Una **domanda d'esame** sarà rappresentare lo stato della memoria della macchina prima del ritorno della funzione chiamata



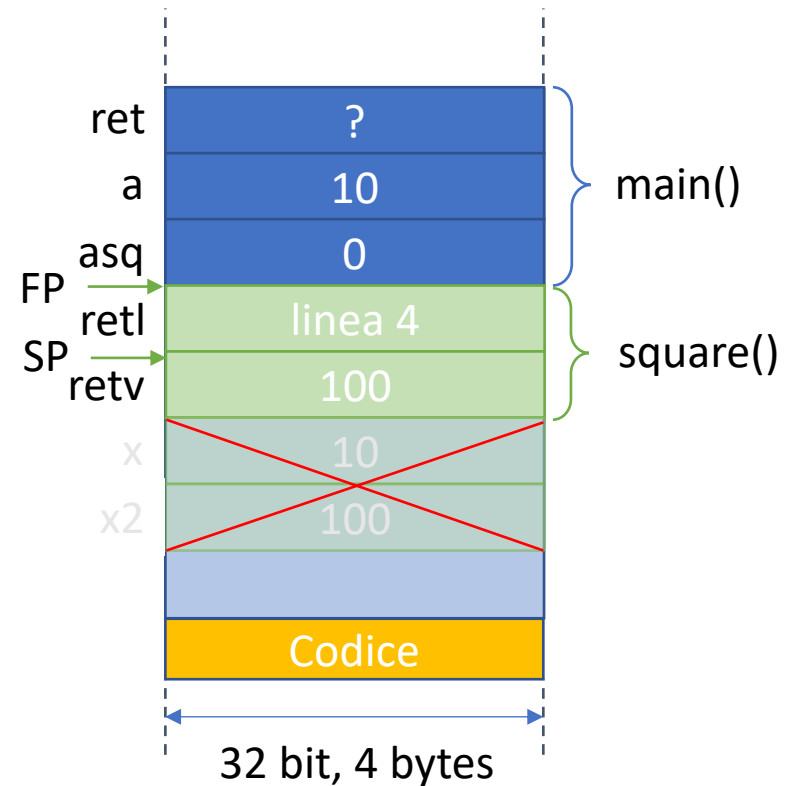
Ritorno da chiamata a funzione

Al ritorno da funzione chiamata a funzione chiamante:

1. SP è incrementato lasciando retv in cima allo stack

Chiamata a funzione *square()*

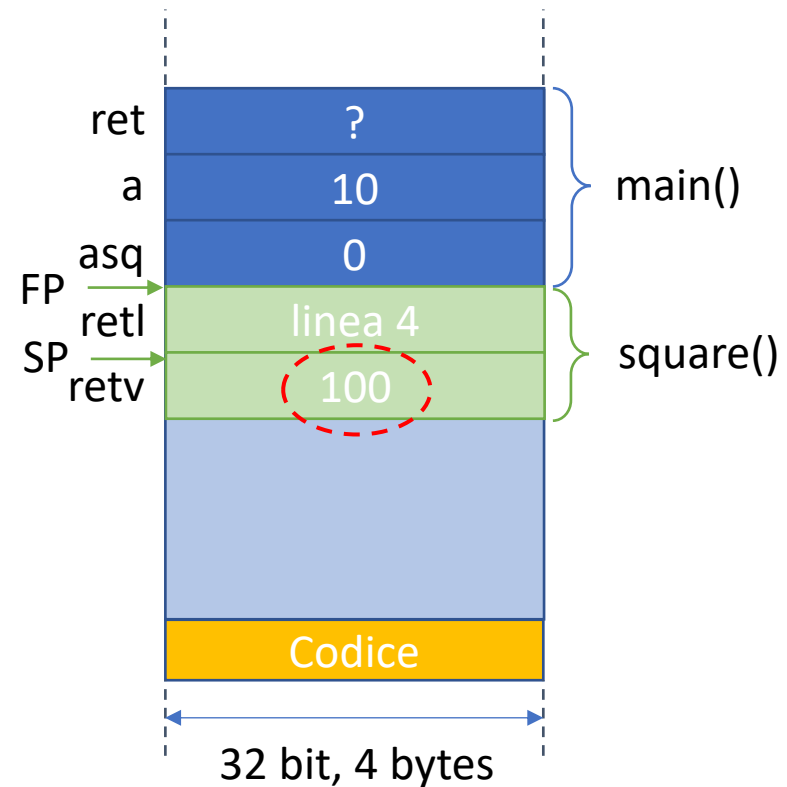
```
1 void main(void) {  
2     int a = 10, asq = 0;  
3     asq = square (a);  
4     printf("%d squared:  
5 %d\n", a, asq);  
6 }  
7  
8 int square (int x) {  
9     int x2 = x * x;  
10    return x2;  
11 }
```



Le variabili *x* ed *x2* sono deallocati dallo stack perchè ormai inutili

Chiamata a funzione *square()*

```
1 void main(void) {  
2     int a = 10, asq = 0;  
3     asq = square (a);  
4     printf("%d squared:  
5 %d\n", a, asq);  
6 }  
7  
8 int square (int x) {  
9     int x2 = x * x;  
10    return x2;  
11 }
```



Il valore da ritornare retv è in cima allo stack (ovvero all'indirizzo più basso)

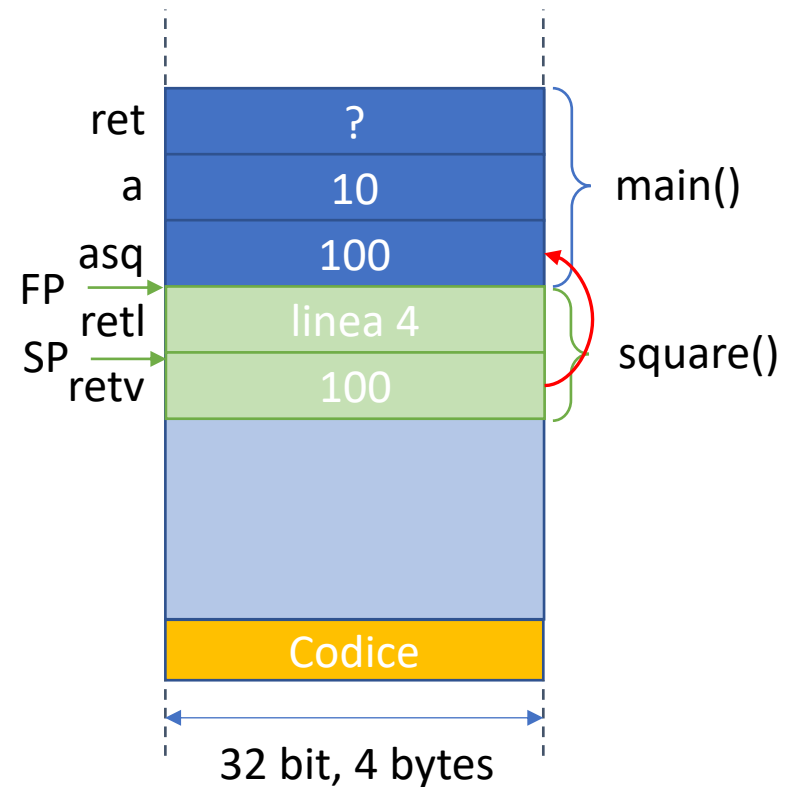
Ritorno da chiamata a funzione

Al ritorno da funzione chiamata a funzione chiamante:

1. SP è incrementato lasciando retv in cima allo stack
2. retv è copiato in asq

Chiamata a funzione *square()*

```
1 void main(void) {  
2     int a = 10, asq = 0;  
3     asq = square (a);  
4     printf("%d squared:  
%d\n", a, asq);  
5 }  
6 int square (int x) {  
7     int x2 = x * x;  
8     return x2;  
9 }
```



Indirizzi di memoria crescenti ↑

L'operatore assegnazione copia il valore in cima allo stack di `retv` è in `asq`

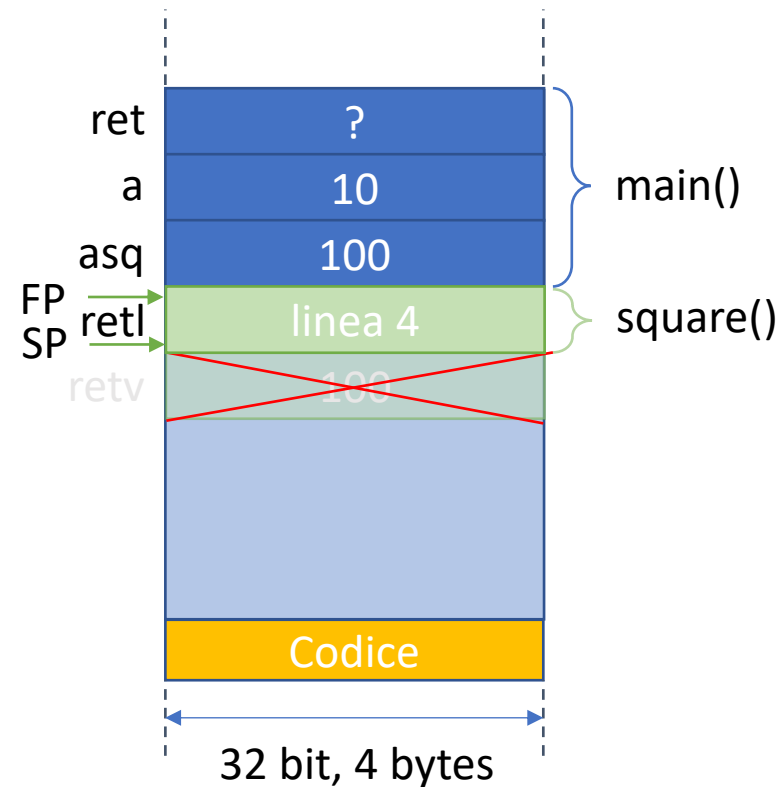
Ritorno da chiamata a funzione

Al ritorno da funzione chiamata a funzione chiamante:

1. SP è incrementato lasciando retv in cima allo stack
2. retv è copiato in asq, SP è incrementato

Chiamata a funzione *square()*

```
1 void main(void) {  
2     int a = 10, asq = 0;  
3     asq = square (a);  
4     printf("%d squared:  
5 %d\n", a, asq);  
6 }  
7  
8 int square (int x) {  
9     int x2 = x * x;  
10    return x2;  
11 }
```



retl è in cima allo stack

Indirizzi di memoria crescenti ↑

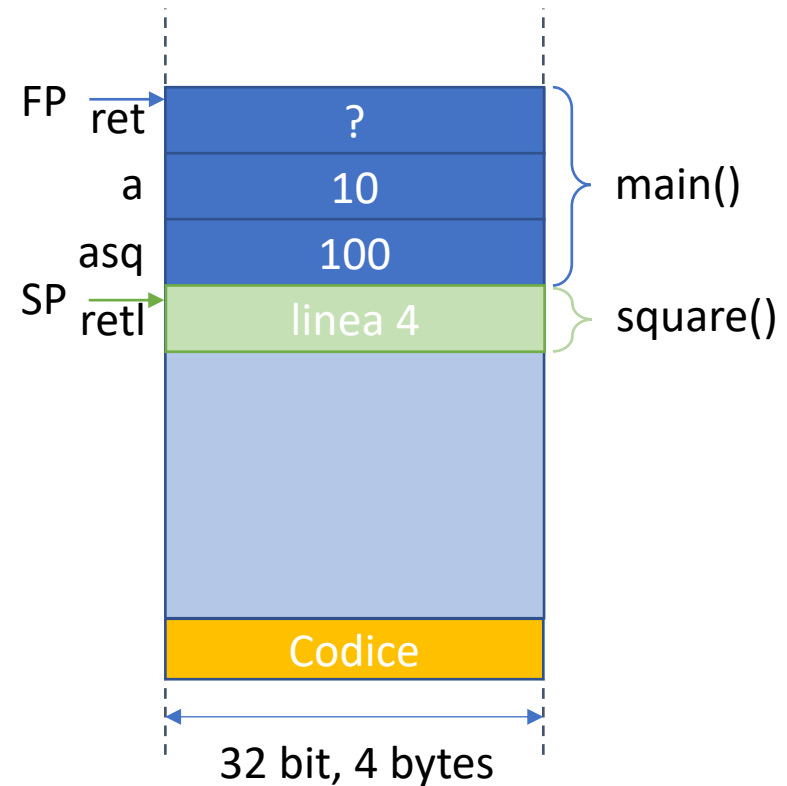
Ritorno da chiamata a funzione

Al ritorno da funzione chiamata a funzione chiamante:

1. SP è incrementato lasciando retv in cima allo stack
2. retv è copiato in asq, SP è incrementato, FP di main è ripristinato

Chiamata a funzione *square()*

```
1 void main(void) {  
2     int a = 10, asq = 0;  
3     asq = square (a);  
4     printf("%d squared:  
%d\n", a, asq);  
5 }  
6 int square (int x) {  
7     int x2 = x * x;  
8     return x2;  
9 }
```



retl è in cima allo stack

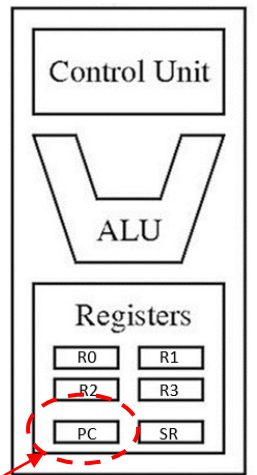
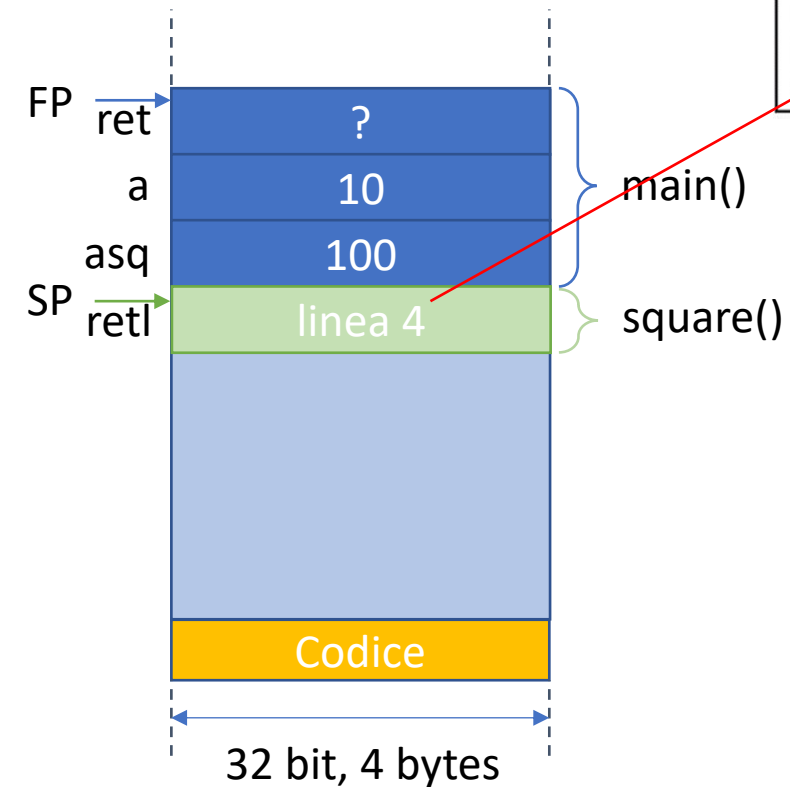
Ritorno da chiamata a funzione

Al ritorno da funzione chiamata a funzione chiamante:

1. SP è incrementato lasciando retv in cima allo stack
2. retv è copiato in asq, SP è incrementato, FP di main è ripristinato
3. Il valore in cima allo stack è copiato in PC

Chiamata a funzione *square()*

```
1 void main(void) {  
2     int a = 10, asq = 0;  
3     asq = square (a);  
4     printf("%d squared:  
5     %d\n", a, asq);  
6 }  
7  
8 int square (int x) {  
9     int x2 = x * x;  
10    return x2;  
11 }
```



Indirizzi di memoria crescenti

La prossima linea eseguita sarà la 4

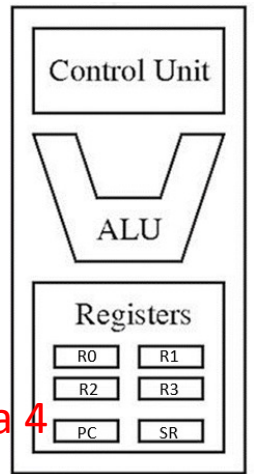
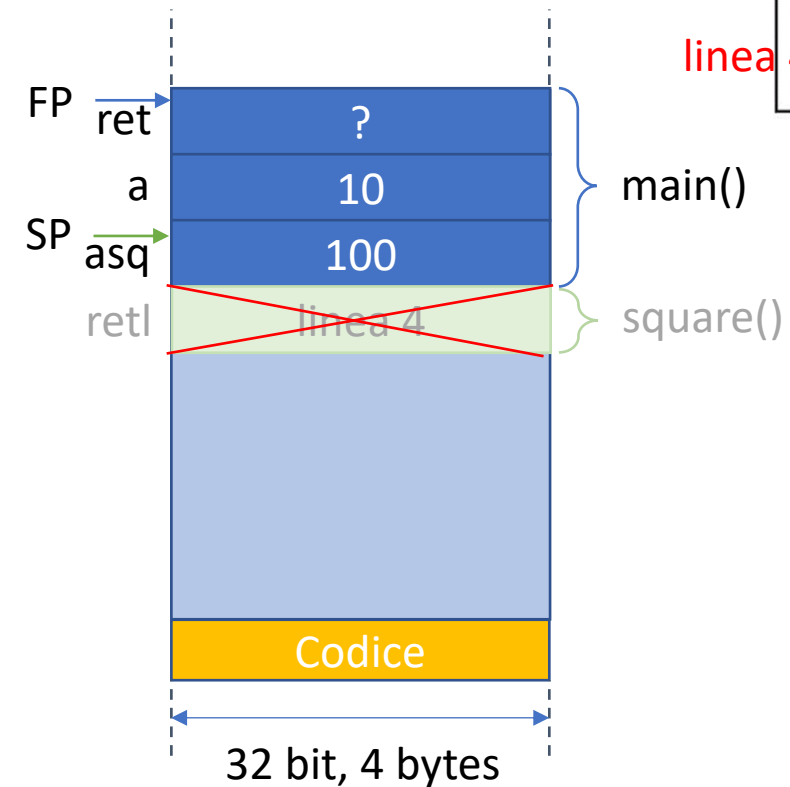
Ritorno da chiamata a funzione

Al ritorno da funzione chiamata a funzione chiamante:

1. SP è incrementato lasciando retv in cima allo stack
2. retv è copiato in asq, SP è incrementato, FP di main è ripristinato
3. Il valore in cima allo stack è copiato in PC
4. SP è incrementato e il frame di *square()* è liberato

Chiamata a funzione *square()*

```
1 void main(void) {  
2     int a = 10, asq = 0;  
3     asq = square (a);  
4     printf("%d squared:  
5     %d\n", a, asq);  
6 }  
7  
8 int square (int x) {  
9     int x2 = x * x;  
10    return x2;  
11 }
```



Indirizzi di memoria crescenti ↑

La prossima linea eseguita sarà la 4

Ritorno da chiamata a funzione

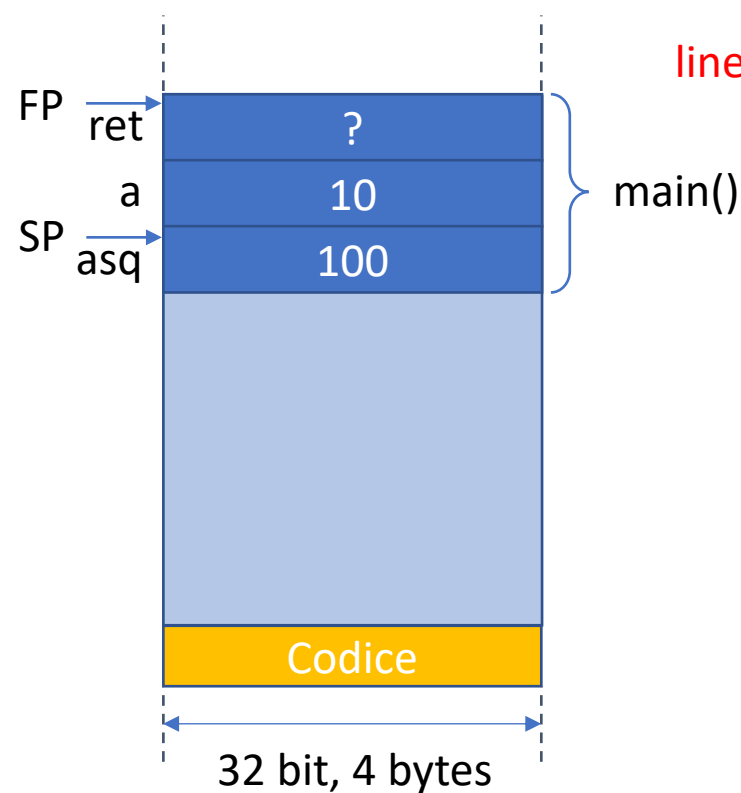
Al ritorno da funzione chiamata a funzione chiamante:

1. SP è incrementato lasciando retv in cima allo stack
2. retv è copiato in asq, SP è incrementato, FP di main è ripristinato
3. Il valore in cima allo stack è copiato in PC
4. SP è incrementato e il frame di *square()* è liberato
5. L'esecuzione del main riprende dalla linea indicata dal PC: la 4

Chiamata a funzione *square()*

```
1 void main(void) {  
2     int a = 10, asq = 0;  
3     asq = square (a);  
4     printf("%d squared:  
5     %d\n", a, asq);  
6 }  
7  
8 int square (int x) {  
9     int x2 = x * x;  
10    return x2;  
11 }
```

PC →



linea 4

L'esecuzione riprende dalla linea 4 del *main()*, ovvero alla *printf()*

Chiamata a funzione - considerazioni

- Il meccanismo di chiamata a funzione illustrato in queste slides è molto ad alto livello e non tiene in conto i dettagli dell'implementazione hardware specifica (RISCV, x86, ARM, etc.)
- Nel corso di Architetture di elaborazione questo modello verrà approfondito (es: salvataggio stato del processore)
- Questo modello è sufficiente a spiegare alcune peculiarità del passaggio di parametri a funzione del linguaggio C